TTTC2453 MACHINE LEARNING

SEMESTER 2 25/26

PROJECT TITLE:

D6 – CREDIT CARD FRAUD DETECTION

LECTURER: DR. WANDEEP KAUR A/P RATAN SINGH

GOOGLE COLAB LINK:

https://colab.research.google.com/drive/1jWLxx0c3h2H25rw0RYQY69xPVneZCCwK?usp=sharing

GROUP MEMBERS:

MATRIC NUMBER	NAME
[redacted]	MUHAMAD ALIF ZULHILMI BIN MOHD ASRI
[redacted]	AHMAD DANISH FARHAN BIN AHMAD ZAWANI
[redacted]	MUHAMMAD AIMAN SYAFIQ BIN MOHD HANAFI

Table of Contents

1.0 PROBLEM FRAMING AND EXPLORATORY DATA ANALYSIS

1.1 Problem Background

This project applies machine learning to credit card fraud detection, classifying each transaction as legitimate (Class 0) or fraudulent (Class 1) based on its transaction features. Fraud detection is an important real-world problem. Fraudulent transactions that go undetected cause financial losses for customers, banks, and payment service providers. However, the task is challenging because fraudulent transactions are very rare compared to legitimate transactions. Legitimate transactions vastly outnumber fraudulent ones, a model can achieve very high accuracy simply by labelling everything as legitimate, while still missing nearly every case of fraud. Therefore, the model must be able to detect a very small number of fraud cases without relying only on overall accuracy. In this project, the modelling process focuses on identifying fraud patterns while handling the severe class imbalance in the dataset.

1.2 Dataset Description

This project uses the Credit Card Fraud Detection dataset, which originally contains 284807 transactions across 31 columns. Of these, Time and Amount are directly interpretable, while V1 through V28 are anonymised numerical features with no disclosed real-world meaning. The target column, Class, takes the value 0 for a legitimate transaction and 1 for a fraudulent one. Most columns are stored as the float64 type, with Class held as an integer. Because V1 to V28 are already numerical, no categorical encoding was required.

1.3 Data Cleaning and Class Distribution

Before model training, the dataset was checked for missing values and duplicate records. The missing value audit showed that there were no missing values in all 31 columns. Therefore, no imputation or row removal was required for missing data. This helped preserve the original transaction information without introducing artificial values into the dataset.

Duplicate records were also checked. It found 1081 repeated rows, which were removed, reducing the dataset from 284807 to 283726 rows. Duplicates were dropped because repeated transactions can bias the model during training and make its evaluation appear more reliable than it really is.

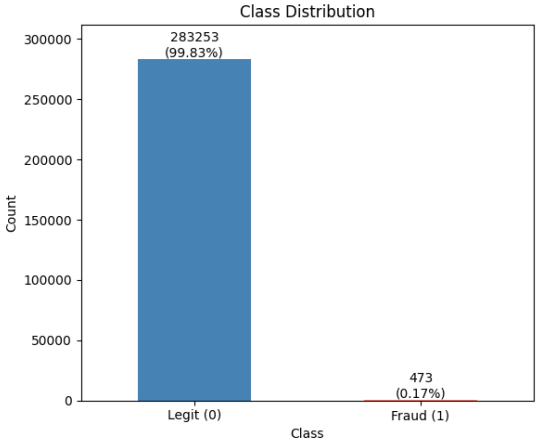

Figure 1 Class Distribution of Legitimate and Fraudulent Transactions

Figure 1 shows that the dataset is extremely imbalanced, with 283253 legitimate transactions and only 473 fraudulent transactions. Legitimate transactions dominate the dataset by 99.83%, while fraudulent transactions represent only 0.17% of the total data. This directly affects the modelling and evaluation strategy. If accuracy is used as the main metric, a model could predict almost every transaction as legitimate and still achieve very high accuracy, but it would fail to detect fraud. Therefore, fraud-sensitive metrics such as recall, F1-score, and AUC-PR are more suitable for evaluating the models.

1.4 Outlier Analysis

Outlier analysis was carried out using the Interquartile Range (IQR) method. Several features showed a substantial share of outliers, the highest being V27 (13.67%), Amount (11.17%), V28 (10.61%), V20 (9.71%), and V8 (8.43%). Rather than remove them, the project retains these values. In fraud detection, an extreme value is not necessarily an error. Fraudulent transactions often look abnormal precisely because they differ from normal spending, so the outliers themselves may carry the signal the model is meant to find. Discarding them would risk discarding that signal. The outliers were therefore kept and handled later through scaling, which limits their influence without erasing them.

1.5 EDA Visualization and Modelling Decisions

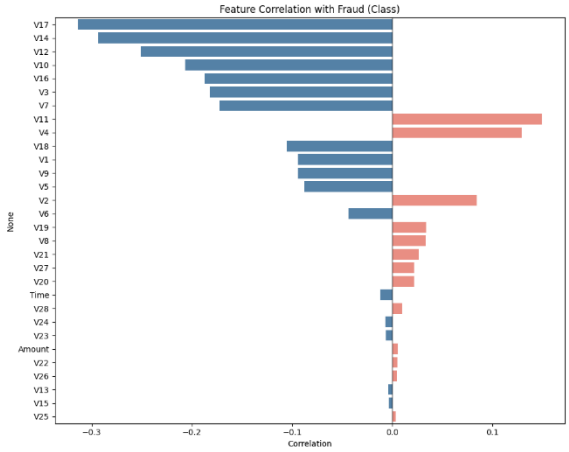

Figure 2 Feature Correlation with Fraud

Figure 2 shows the features that have the strongest relationship with the target variable Class. The most important features based on correlation were V17, V14, V12, V10, and V16. These features showed stronger relationships with fraud compared to other variables. However, no single feature had a perfect relationship with the fraud class. This means that fraud detection cannot depend on only one feature. Instead, the model needs to learn patterns from several features together. This supported the decision to compare different model types, including a baseline model, an ensemble model, and a deep learning model.

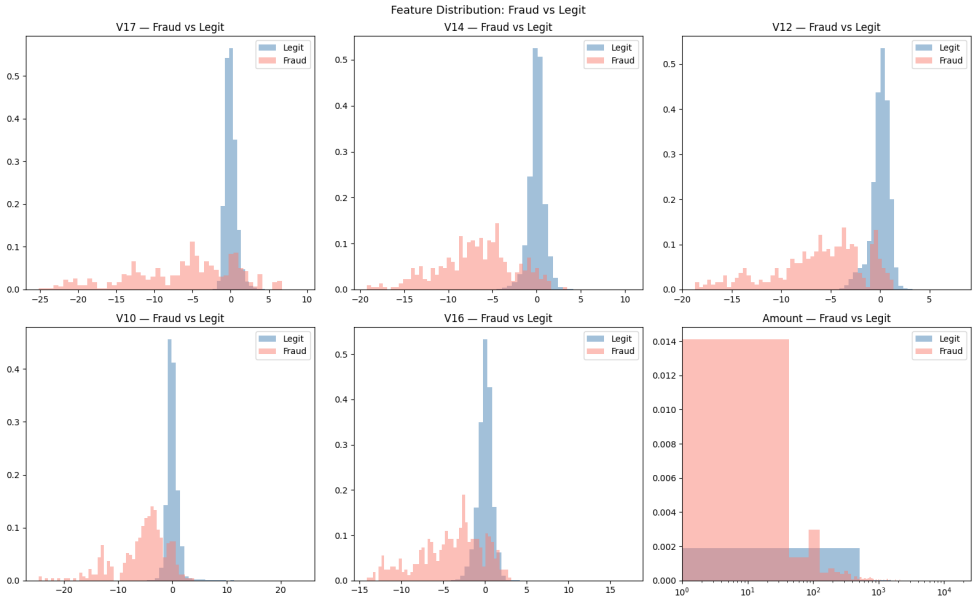

Figure 3 Feature Distribution of Fraud and Legitimate Transactions

Figure 3 compares the distribution of selected features between legitimate and fraudulent transactions. The selected features include V17, V14, V12, V10, V16 and Amount. The plot shows that fraud transactions have different patterns compared to legitimate transactions. In several features, fraud values are more spread out and shifted away from the normal transaction region. This shows that the selected features contain useful fraud-related information. However, there is still overlap between the two classes, which makes the classification task difficult. Therefore, machine learning models are needed to learn the combined patterns from multiple features.

1.6 Feature Engineering

Feature engineering was applied to improve the quality of the input features before model training. The Time feature was converted into two cyclical features, Hour_sin and Hour_cos, because time follows a repeating daily pattern. This helps the model understand that hour 23 and hour 0 are close to each other, instead of treating them as far apart. The Amount feature was also transformed using log1p(Amount) because the original amount distribution was highly right skewed. The skewness was reduced from 16.98 to 0.16 after transformation, making the feature more stable for modelling. The raw columns Time, Hour, and Amount, were removed to avoid redundant features.

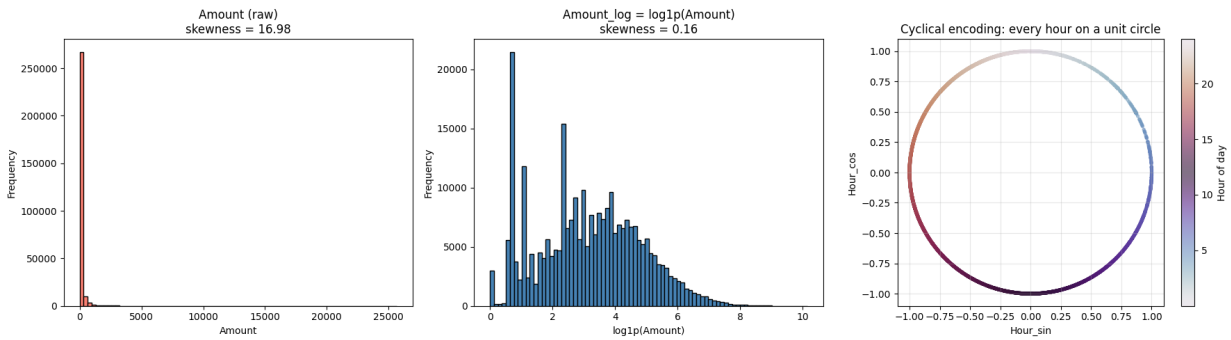

Figure 4 Amount Distribution Before and After Log Transformation

1.7 Train-Test Split, Scaling and Imbalance Handling

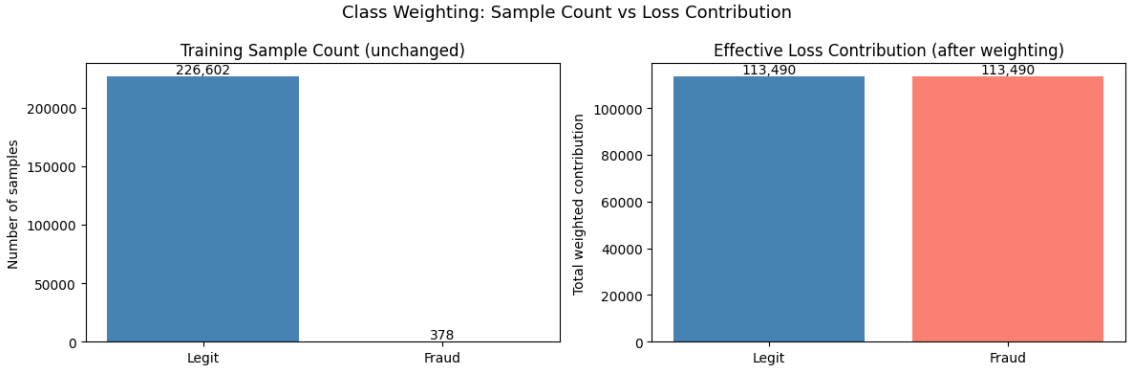

Figure 5 Effect of Class Weighting on Training Sample Count and Loss Contribution

The dataset was split into training and test sets using an 80:20 ratio with stratified sampling. Stratified sampling was used to maintain a similar proportion of legitimate and fraudulent transactions in both sets, since the dataset is highly imbalanced. The final training set contained 226,980 records, while the test set contained 56,746 records.

RobustScaler was applied because the dataset contains outliers, especially in the transaction amount and several numerical features. The scaler was fitted only on the training set and then applied to the test set to avoid data leakage.

To handle the class imbalance, class weighting was used during model training. The legitimate class received a weight of 0.5008, while the fraud class received a much higher weight of 300.2381. This allows the model to give stronger attention to fraud cases without changing the original number of samples.

2.0 MODEL PRINCIPLE

2.1 Model 1: Baseline Model (Logistic Regression)

Logistic regression works by drawing a straight dividing line between classes in the feature space. It takes all the input features, multiplies each by a learned weight, sums them up, and passes the result through a sigmoid function to produce a probability between 0 and 1. Every time the model makes a wrong or uncertain prediction, it adjusts its weights through gradient descent to do better next time. For this dataset, logistic regression is a solid baseline choice. The V1–V28 features are already PCA-transformed, meaning they are decorrelated and continuous, which suits the model well. The coefficients are also easy to interpret, each one shows how much a feature pushes the prediction toward fraud or not. The main weakness is that it cannot detect feature interactions or non-linear patterns. If fraud only appears when two features combine in a specific way, the model will miss it entirely. This makes it useful as a starting point but limited beyond that.

2.2 Model 2: Ensemble / Intermediate Model (XGBoost)

XGBoost builds a series of small decision trees one after another, where each new tree focuses on fixing the mistakes made by the ones before it. Rather than learning everything at once, it gradually improves by targeting whatever errors remain after each round. The loss function combines logistic loss with regularisation, which keeps individual trees from becoming too complex. Because the model splits

data based on feature thresholds, it naturally picks up on non-linear patterns and interactions between features, something a simple linear model cannot do. It also requires no feature scaling or linearity assumptions, making it more flexible overall. The key settings to tune are the number of trees, tree depth, learning rate, and subsampling rates. The scale_pos_weight parameter is especially useful here for handling the class imbalance by giving more weight to fraud cases during training. The main downsides are the risk of overfitting if not properly tuned and the fact that predictions are harder to interpret compared to simpler models.

2.3 Model 3: Deep Learning Model (Multilayer Perceptron)

The MLP is a feed-forward neural network that takes the 31 input features and passes them through two hidden layers. First 32 neurons, then 16, before a final sigmoid neuron outputs the fraud probability. Each hidden layer applies a linear transformation followed by a ReLU activation, which allows the network to learn complex, non-linear patterns across features in a layered way. Training uses the Adam optimiser with backpropagation to minimise binary cross-entropy loss. Class weights are applied so that the rare fraud examples contribute more strongly to each weight update. Dropout at a rate of 0.2 is added after each hidden layer to prevent the network from becoming too reliant on specific neurons, while early stopping monitors validation loss to avoid overfitting. The main limitation here is data scarcity. With only 378 fraud examples available for training, the network has very little minority-class signal to learn from. Neural networks generally need large amounts of data to perform well, making this a significant challenge.

3.0 MODEL IMPLEMENTATION

3.1 Experimental Setup

Table 1 Experimental Setup Configuration

Item	Setting
Languages	Python 3
Libraries	Pandas, NumPy, scikit-learn, imbalanced-learn (SMOTE trial), XGBoost, TensorFlow/Keras, Matplotlib, Seaborn
Train-test Split	80/20 stratified on Class
Random Seed	42
Primary Metric	AUC-PR (average precision)
Secondary Metric	Recall, Precision, F1 (fraud class), confusion matrix

3.2 Preprocessing Pipeline

Every model works with the same set of 31 input features: V1 through V28, Hour_sin, Hour_cos, and Amount_log. Before splitting the data, stateless transformations were applied, which are cyclical hour encoding and log1p scaling on the Amount column. However, the RobustScaler was fitted strictly on the training set and then applied to the test set, ensuring no test-set information leaked into the training process.

Logistic regression and the MLP were trained on the scaled features, while XGBoost used the unscaled version. This is because tree-based models rely on split thresholds rather than absolute values,

so feature scaling has no real effect on them. To address the class imbalance in the dataset, all models used loss reweighting rather than altering the data itself. Logistic regression and the MLP used class_weight='balanced', while XGBoost was given a scale_pos_weight of 599.48, which corresponds to the legitimate-to-fraud ratio in the training set. The SMOTE experiment discussed in Section 4.7 followed the same principle, that is, oversampling was only done on the training data, and the test set remained untouched at every stage.

3.3 Baseline Model Implementation

Table 2 Baseline Model (Logistic Regression) Configuration

Item	Description
Model	LogisticRegression (scikit-learn)
Key Hyperparameters	class_weight='balanced', solver='lbfgs', max_iter=1000, random_state=42; default decision threshold 0.5
Preprocessing	RobustScaler - scaled 31 features
Training Method	Full-batch L-BFGS optimisation of weighted log-loss on 226,980 training rows
Output	Fraud probability; coefficients inspected for interpretability
Metric Used	AUC-PR (primary), recall/precision/F1/ROC-AUC (secondary)

This baseline is considered fair because it shares the same feature set, train-test split, random seed, and imbalance handling as the models that follow. Any difference in performance can therefore be credited to model capacity alone, rather than any data-related advantage. The fitted intercept was −3.84, indicating the model leans toward predicting a transaction as legitimate by default and only shifts toward fraud when the relevant features move strongly in the direction their coefficients indicate. The most influential were V4 (+1.66), V11 (+0.95), and V1 (+0.93), where higher values push toward fraud, alongside V12 (−1.37), V14 (−1.28), and V10 (−1.27), where lower (more negative) values push toward fraud.

3.4 Ensemble / Intermediate Model Implementation

XGBoost was tuned using RandomizedSearchCV, testing 10 candidate configurations across 5-fold stratified cross-validation, 50 fits in total, with average precision as the scoring metric. A fixed-seed StratifiedKFold was used throughout to make sure every candidate was evaluated on the same folds. The XGBoost achieved a mean CV AUC-PR of 0.8474, while the best configuration found by the search came in at 0.8452, with a difference of just −0.0021. Given that the fold-to-fold standard deviation was around 0.030, this gap is well within normal variation and essentially amounts to a statistical tie. In other words, the default settings were already close to optimal for this dataset.

Table 3 XGBoost Hyperparameter Search

Hyperparameter	Value Tested	Best Value
n_estimators	100, 200, 300	200

To be continued…

…continuation		
max_depth	3, 4, 5	4
learning_rate	0.01, 0.05, 0.1	0.1
subsample	0.8, 1.0	1.0
colsample_bytree	0.8, 1.0	0.8

3.5 Deep Learning Model Implementation

Table 4 Deep Learning Model (MLP) Architecture and Training Configuration

Layer / Component	Setting
Input Layer	31 Features (RobustScaler - scaled)
Hidden Layers	Dense(32, ReLU) + Dropout(0.2) → Dense(16, ReLU) + Dropout(0.2)
Output Layer	Dense(1, sigmoid)
Total Parameters	1569 Trainable
Loss Function	Binary Cross-Entropy with class weights (legit 0.5008, fraud 300.24)
Optimiser	Adam (default learning rate)
Epochs / Batch Size	15 epoch, batch size 2048, validation_split = 0.2
Regularization	Dropout 0.2; EarlyStopping (monitor = val_loss, patience = 10, restore_best_weights) that did not trigger within 15 epochs
Decision Threshold	Default 0.5

4.0 PERFORMANCE ANALYSIS

4.1 Metric Justification

With fraud making up only 0.17% of transactions, accuracy is essentially meaningless here, a model that labels everything as legitimate would still score 99.83% while catching no fraud whatsoever. AUC-PR is therefore used as the primary metric, as it summarises the precision-recall trade-off across all thresholds and is widely recommended for imbalanced binary classification problems.

ROC-AUC is included as a secondary reference only. The sheer size of the legitimate class keeps the false-positive rate artificially low, which makes every model appear stronger on ROC than it might be. Recall, precision, and F1 for the fraud class are also reported, though it's worth noting that these reflect performance at a single operating point, the default 0.50 threshold, rather than the model's overall capability.

4.2 Overall Results Comparison

The table below reports all metrics on the same held-out test set at the default 0.50 threshold (AUC-PR and ROC-AUC are threshold-independent).

Table 5 Overall Model Performance Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC	AUC-PR	Notes
Logistic Regression	0.9750	0.0558	0.8737	0.1049	0.9636	0.6807	class_weight= 'balanced'
XGBoost	0.9992	0.7576	0.7895	0.7732	0.9674	0.8093	scale_pos_weight
MLP	0.9794	0.0661	0.8632	0.1228	0.9608	0.6614	Class-weighted, 15 epochs

4.3 Baseline Model Result Interpretation

The class-weighted logistic regression achieves strong recall at 0.8737, catching 83 out of 95 frauds, but this comes at a steep cost, which is 1404 false alarms and a precision of just 0.0558, pulling the F1 down to 0.1049. This is largely expected behaviour for a linear model pushed to be aggressive on a rare class. A single straight boundary simply cannot tightly enclose the fraud region, so reaching high recall inevitably sweeps in a large portion of legitimate transactions alongside it.

That said, its AUC-PR of 0.6807 sits well above the 0.17% prevalence baseline, meaning the model has genuinely picked up on fraud signal, it just lacks the flexibility to translate that signal into precise decisions. It holds up well enough as a reference point, even if it falls short as a practical detector.

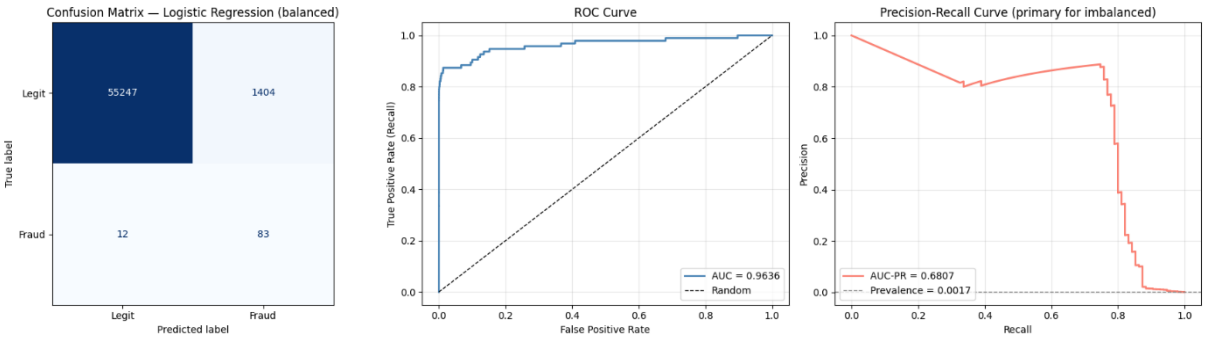

Figure 6 Logistic Regression: confusion matrix, ROC curve, and precision–recall curve

4.4 Ensemble Model Result Interpretation

XGBoost improves on the baseline across every meaningful measure. Its AUC-PR reaches 0.8093, and at the default threshold, it achieves a precision of 0.7576 and a recall of 0.7895, giving an F1 of 0.7732, a much more balanced operating point compared to the recall-at-all-costs behaviour of the linear models. The confusion matrix makes the difference concrete: 75 frauds caught, 20 missed, and only 24 false alarms, against the baseline's 1,404.

The improvement comes from the ensemble's ability to model non-linear feature interactions across many shallow trees, allowing it to draw a tight boundary around the fraud region rather than relying on a single hyperplane. The feature importance plot reflects this, showing the model leans heavily on a handful of PCA components, V14 alone accounts for roughly 35% of total importance,

followed by V12 and V10. There's also no obvious sign of overfitting, as the gap between the cross-validated and test AUC-PR remains small.

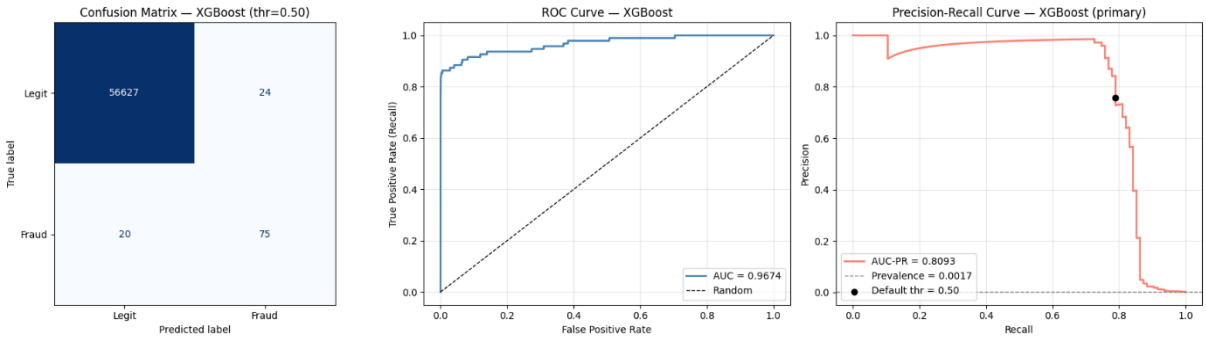

Figure 7 XGBoost: confusion matrix, ROC curve, and precision–recall curve (default 0.50 threshold)

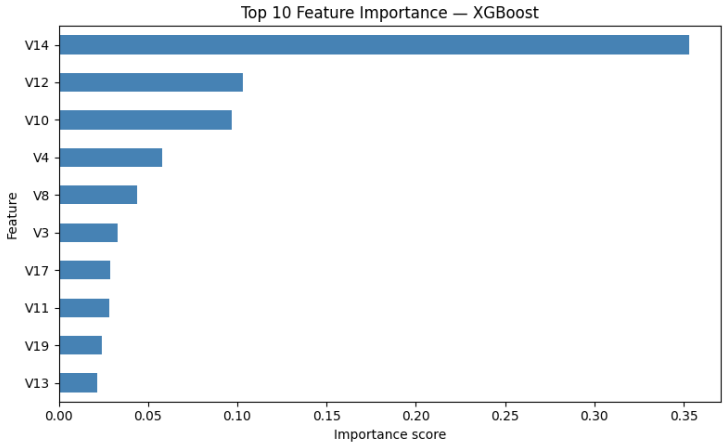

Figure 8 Top 10 XGBoost feature importances: V14, V12, and V10 dominate

4.5 Deep Learning Model Result Interpretation

Despite being the most complex of the three models, the MLP actually underperforms XGBoost on the primary metric, coming in at an AUC-PR of 0.6614. At the default threshold, its behaviour looks much like the linear baseline, with a recall of 0.8632 and a precision of just 0.0661, catching 82 frauds but generating 1,158 false alarms in the process.

The loss curves don't point to overfitting; both training and validation loss decrease smoothly throughout, with no upward turn on the validation side. The network simply failed to find a better representation than the gradient-boosted trees. This isn't too surprising given the nature of the data, that the features are already PCA components on a modest-sized tabular dataset, which is precisely the setting where tree ensembles tend to have the upper hand over small neural networks. A deeper or more complex network would likely need considerably more data and careful tuning to justify the added complexity.

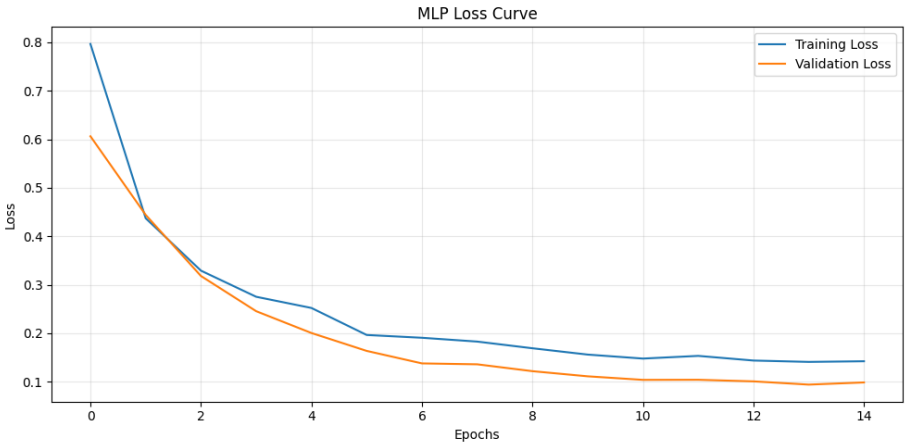

Figure 9 MLP training and validation loss across epochs. Both decrease smoothly with no upward turn in validation loss

4.6 Confusion Matrix / Error Analysis

The confusion matrices help put the real-world cost structure into perspective. In fraud detection, the two types of errors are not equal. A false negative, where a fraudulent transaction slips through undetected, carries a much higher cost than a false positive, where a legitimate transaction gets flagged for review. The table below summarises how each model performed across all four outcomes on the test set.

Table 6 Confusion Matrix Summary

Model	TP (fraud caught)	FN (fraud missed)	FP (false alarm)	TN (legit passed)
Logistic Regression	83	12	1404	55247
XGBoost	75	20	24	56627
MLP	82	13	1158	55493

The class-weighted models, logistic regression, and the MLP, miss the fewest frauds at 12 and 13, respectively, but generate over a thousand false alarms each in return. In practice, this is difficult to work with, as analysts would constantly be sifting through large volumes of clean transactions to find actual fraud.

XGBoost strikes a more deployable balance. It misses a few more frauds at 20 but produces only 24 false alarms — a far more manageable workload for a review team. It's also worth noting that these counts reflect the default 0.50 threshold only. Since XGBoost has the highest AUC-PR, its threshold can be lowered to recover more fraud while keeping false positives well below what the other two models produce.

The errors that persist across all models share a common cause, that these are fraud cases whose feature values fall within the legitimate transaction cloud, as seen in the EDA boxplots where the two classes overlap heavily. No threshold adjustment can cleanly separate them.

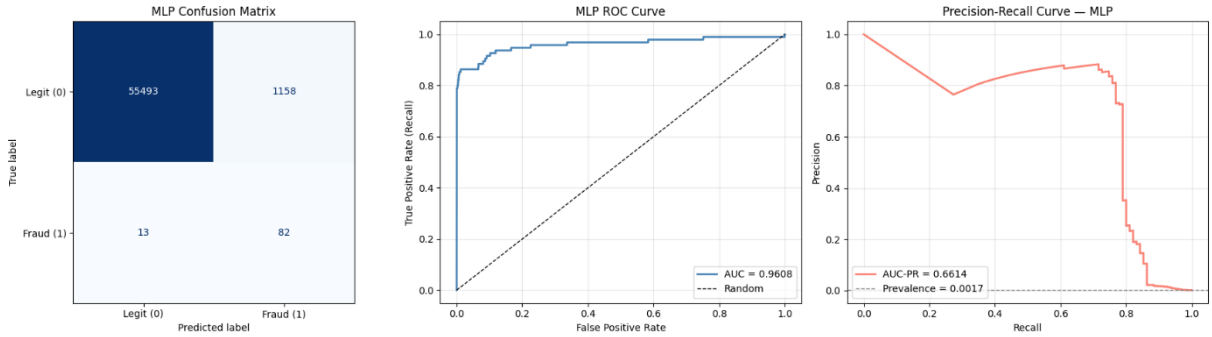

Figure 10 MLP: confusion matrix, ROC curve, and precision–recall curve.

4.7 Effect of Imbalance Handling

Imbalance handling was studied most directly on XGBoost by retraining the best configuration with and without scale_pos_weight (same threshold, same test set).

Table 7 Effect of Class Weighting on XGBoost

XGBoost Variant	AUC-PR	Recall	Precision
Without weighting	0.7940	0.7158	0.9577
With scale_pos_weight	0.8093	0.7895	0.7576

Up-weighting the fraud class lifted AUC-PR (0.7940 → 0.8093) and recall (0.7158 → 0.7895), at the cost of precision (0.9577 → 0.7576). This is the standard trade-off: the model now flags more transactions, catching more fraud but also producing more false alarms. Because missing fraud is the costly error here, the gain in recall and the higher AUC-PR justify the weighting. For Logistic Regression, the earlier comparison showed the same direction, weighting raised recall from 0.59 to 0.87 while precision collapsed, and showed that SMOTE offered no advantage over simple class weighting, which is why weighting was adopted across the pipeline.

4.8 Summary of Model Failure Modes

Table 8 Summary of Model Failure Modes

Model	Main Strength	Main Weakness	Likely Reason
Logistic Regression	Very high recall. Simple and fully interpretable	Extremely low precision (1,404 false alarms)	A single linear boundary cannot enclose the fraud region tightly
XGBoost	Best AUC-PR; balanced precision and recall; few false alarms	Misses slightly more fraud at the 0.50 threshold (20 FN)	Conservative default threshold; less interpretable than LR

To be continued…

continuation…

MLP	High recall; healthy, stable training	Lowest AUC-PR; very low precision; added complexity	A small tabular PCA dataset favours tree ensembles over a small network

Overall, XGBoost stands out as the strongest model for this task, leading on the metric that matters most under class imbalance. That said, the deployment threshold shouldn't be treated as fixed. In practice, it should be calibrated against the bank's actual cost trade-off between letting a fraud slip through and triggering an unnecessary investigation.

5.0 SUITABILITY JUSTIFICATION

Based on the final model comparison, XGBoost was selected as the most suitable model for this credit card fraud detection problem. This decision was not based on accuracy only because the dataset is highly imbalanced, where fraud cases represent only 0.17% of the total data. In this situation, accuracy can be misleading because a model may get high accuracy by predicting most transactions as legitimate. Therefore, metrics such as precision, recall, F1-score, and AUC-PR are more important for evaluating the model.

XGBoost showed the best overall balance between detecting fraud and reducing false alarms. It achieved a fraud precision of 0.7576, recall of 0.7895, F1-score of 0.7732, ROC-AUC of 0.9674, and AUC-PR of 0.8093. These results show that XGBoost was able to detect a good number of fraud cases while keeping the number of wrong frauds alerts low. This is important because too many false alerts can affect legitimate customers and increase the workload for manual checking.

XGBoost is also suitable for this dataset because the data is tabular and contains many numerical features. From the EDA, no single feature was strong enough to separate fraud and legitimate transactions perfectly. This means the model needs to learn patterns from several features together. XGBoost is suitable for this because it can capture non-linear relationships and interactions between features better than Logistic Regression. Although MLP can also learn non-linear patterns, its fraud precision was much lower, and it produced too many false positives.

The main trade-off in this task is between catching more fraud cases and reducing false positives. Logistic Regression and MLP achieved higher recall, but they also wrongly flagged many legitimate transactions as fraud. Logistic Regression produced 1,404 false positives, while MLP produced 1,158 false positives. In comparison, XGBoost produced only 24 false positives, which makes it more practical and reliable. Although XGBoost missed 20 fraud cases, it still gave the best balance between precision, recall, F1-score, and practical usability. Therefore, XGBoost is the most suitable model for this credit card fraud detection problem.

6.0 IMPROVEMENT MECHANISM

The first improvement is to tune the decision threshold of the XGBoost model. Based on the final result, XGBoost achieved the strongest AUC-PR score of 0.8093 and a good fraud F1-score of 0.7732. However, it still missed 20 actual fraud transactions. This means some fraud cases were predicted as legitimate, which is risky in a fraud detection system. This may happen because the current threshold does not fully prioritise fraud recall. To address this issue, the threshold can be adjusted using the precision-recall curve. A lower threshold may help the model detect more fraud cases and reduce false negatives, but it must be selected carefully so that false positives do not increase too much.

The second improvement is to create more behaviour-based features. The current dataset mainly uses anonymised numerical features, cyclical time features, and log-transformed transaction amount. However, some fraud cases may still be missed because these features may not fully describe real suspicious behaviour. For example, fraud may be related to sudden changes in spending amount, repeated transactions in a short time, unusual transaction hours, or very short gaps between transactions. Creating these additional features can give the model more meaningful information about abnormal transaction behaviour. This may help the model separate fraud and legitimate transactions more clearly.

The third improvement is to optimise the class weight or cost-sensitive learning setting. In this project, class weighting was used because the fraud class was very small. However, the results still showed a trade-off between false negatives and false positives. XGBoost missed 20 fraud cases, while Logistic Regression and MLP produced many false positives. This shows that the current weighting may not be fully balanced for the real cost of fraud detection. To improve this, different class-weight values can be tested instead of using only the automatically computed weight. For example, several fraud weights can be compared to find the setting that gives better recall without increasing false positives too much. This improvement would make the model training more aligned with the actual cost of fraud detection, where missing a fraud case and wrongly flagging a legitimate customer have different impacts.

7.0 REFERENCES

Machine Learning Group, Université Libre de Bruxelles. (2018). *Credit Card Fraud Detection Dataset*. Kaggle. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud (accessed via TensorFlow mirror: https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv)

8.0 APPENDIX

Appendix A AI Tool Usage Disclosure

Tools used: Gemini (Google), Claude (Anthropic), ChatGPT (OpenAI)

How it was used (code generation only):

- Data pipeline is generated with the purpose to audit missing values, duplicate checking, outline IQR analysis and class distribution plots
- Writing feature engineering code for cyclical time encoding (Hour_sin, Hour_cos) and log1p transformation of Amount
- Producing sklearn model training code for Logistic Regression and XGBoost, including the SMOTE vs class weight comparison
- Build the TensorFlow/ Keras MLP architecture, training loop, and EarlyStopping callback
- Generated the evaluation and visualisation code (confusion matrices, ROC curves, PR curves, feature importance charts)

What AI was not used for:

- All the Interpretation sections in the notebook were written by the group
- All written sections in this report (Section 1-7) were produced independently by the group
- Modelling decision such as choosing class weighting over SMOTE, retaining outliers, and selecting AUC-PR as the primary metric reflect the group's own reasoning